Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО».

Факультет программной инженерии и компьютерной техники

Тестирование программного обеспечения
Лабораторная работа №1
Вариант 7015

Выполнил: Путинцев Данил
Группа P3307
Проверил(а)
Преподаватель: Бострикова Дарья

Санкт-Петербург
2026 год

Оглавление

Задание к лабораторной работе 3
Ход работы 4
Задание 1. Математический анализ функции 4
Задание 1. Программные ограничения для функции 6
Задание 2. 7
Функция Insert для чисел 7
Функция Insert для строк 7
Функция Delete 8
Функция Find 8
Задание 3. Доменная модель для заданного текста и тестовое покрытие для неё ... 9
ApproachActionTest (“Приближение”) 10
CountActionTest (“Подсчет чисел”) 10
RepeatActionTest (“Повтор текста”) 10
DomainModelTest 10
Исходный код или ссылка на него 10
Вывод 11

Задание к лабораторной работе

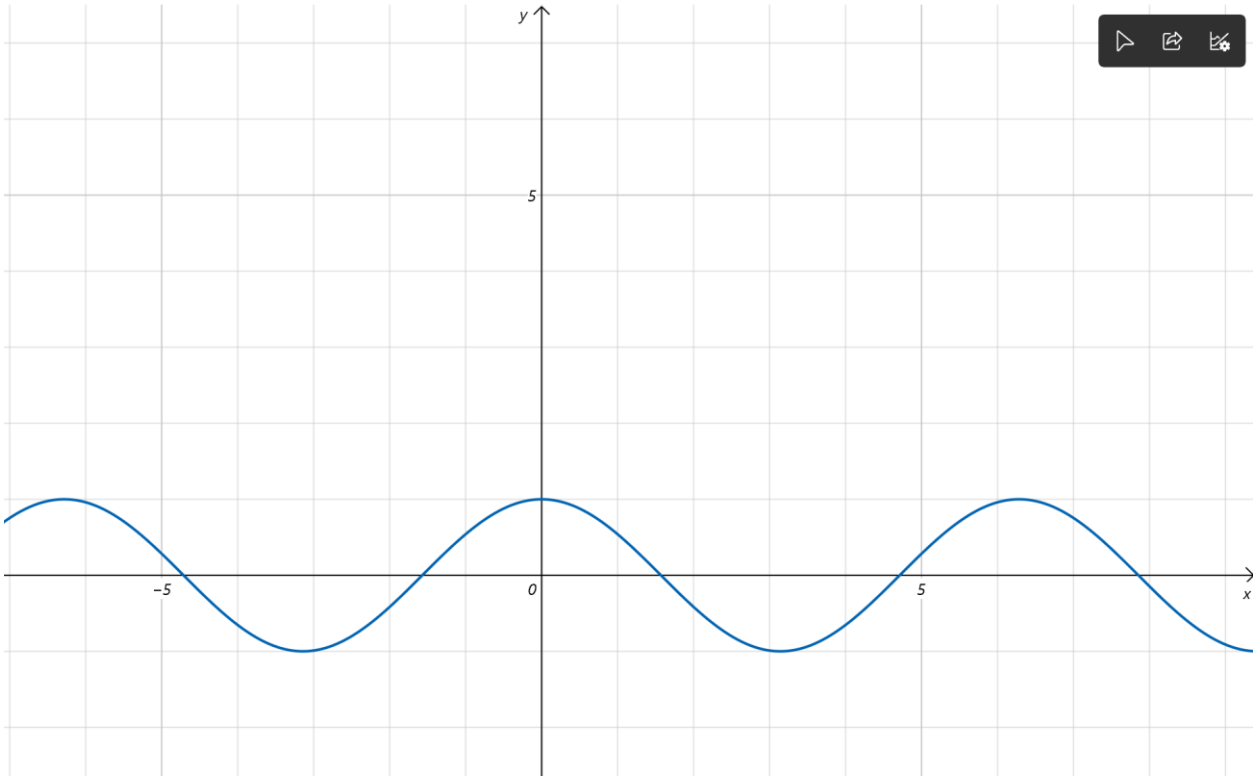
1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

1. Функция cos(x)
2. Программный модуль для работы с хеш-таблицей с закрытой адресацией (Hash Integer, http://www.cs.usfca.edu/~galles/visualization/OpenHash.html)
3. Описание предметной области: Форд продолжал считать вслух. Это одно из самых агрессивных действий, которые вы можете применить к компьютеру, равносильное тому, чтобы медленно приближаться в темноте к человеку, повторяя: "Умри... умри... умри..."

Ход работы.

Задание 1. Математический анализ функции

Рассмотрим функцию y = cos(x)



Заметим, что значение функции $y \in [-1;\ 1]$, а аргумент функции x $\in [-\infty;\ +\infty]$

Также заметим, что функция периодическая, поэтому можно рассмотреть один период функции и затем сделать смещение на величину этого периода. Период функции y = cos(x) равен 2π. Рассмотрим промежуток для аргумента функции: x $\in [0;\ +2\pi]$

Выделим точки, значение которых общеизвестно:

x	0	$\frac{\pi}{6}$	$\frac{\pi}{4}$	$\frac{\pi}{3}$	$\frac{\pi}{2}$	$\frac{2\pi}{3}$	$\frac{3\pi}{4}$	$\frac{5\pi}{6}$	π
y	1	$\frac{\sqrt{3}}{2}$	$\frac{\sqrt{2}}{2}$	0.5	0	-0.5	$-\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{3}}{2}$	-1

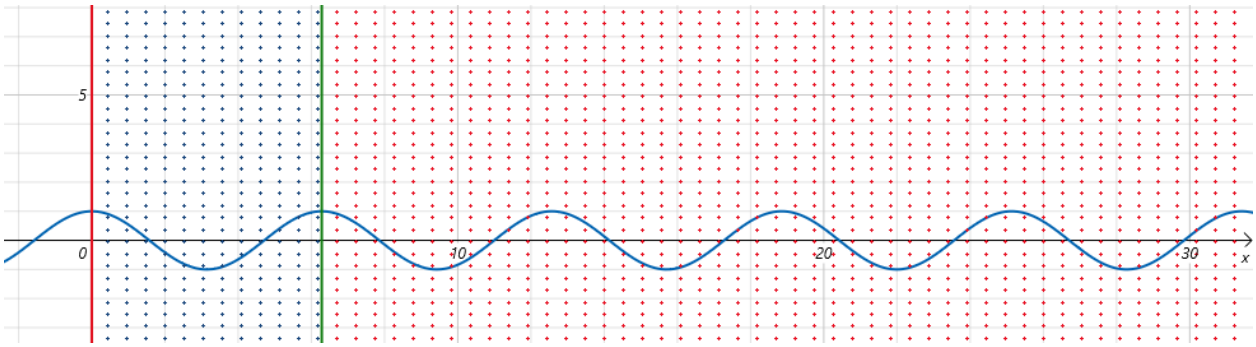
x	$\frac{7\pi}{6}$	$\frac{5\pi}{4}$	$\frac{4\pi}{3}$	$\frac{3\pi}{2}$	$\frac{5\pi}{3}$	$\frac{7\pi}{4}$	$\frac{11\pi}{6}$	2π
y	$-\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{2}}{2}$	-0.5	0	0.5	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1

Рассмотренная область от 0 до 2pi

Теперь применим свойство периодичности для составления тестового покрытия:

cos(x) = cos(x + 2π)

x	$\frac{19\pi}{6}$	$\frac{85\pi}{4}$	$\frac{37\pi}{3}$	$\frac{30000\pi}{2}$	$\frac{37040\pi}{3}$	$\frac{214103\pi}{4}$	$\frac{416603\pi}{6}$	857238π
y	$-\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{2}}{2}$	-0.5	0	0.5	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1



Рассмотренная область от 0 до +inf

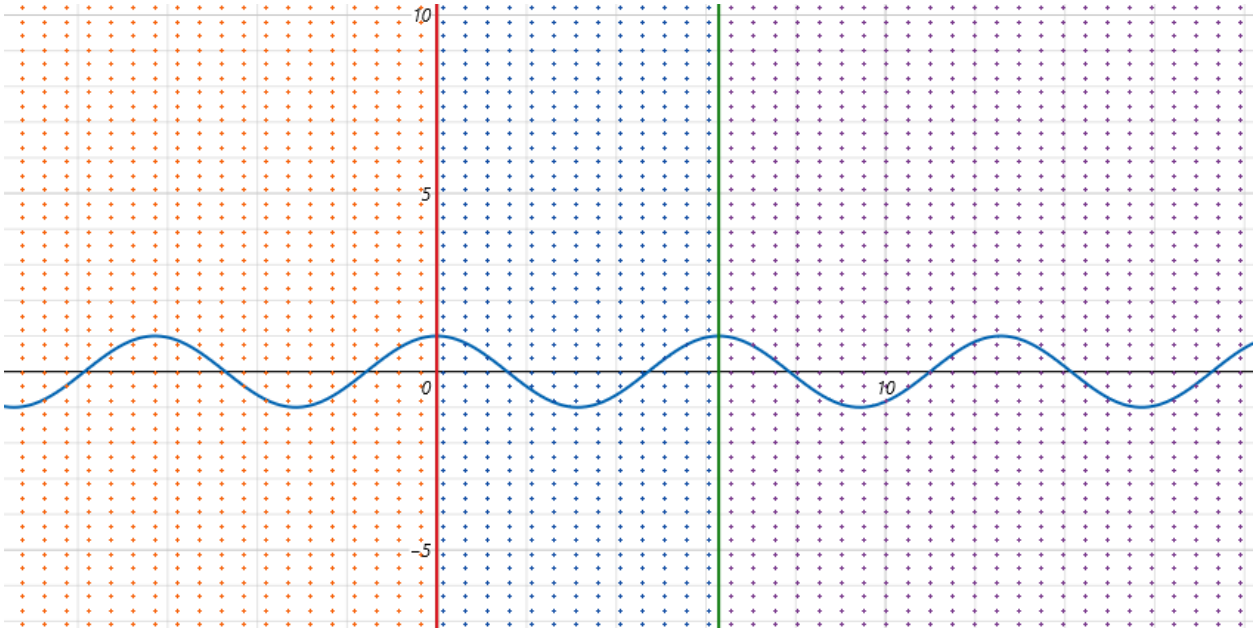
Заметим, что функция cos(x) – четная, а это означает cos(x) = cos(-x), а это значит, что мы можем рассмотреть отрицательные аргументы для функции

x	0	$-\frac{\pi}{6}$	$-\frac{\pi}{4}$	$-\frac{\pi}{3}$	$-\frac{\pi}{2}$	$-\frac{2\pi}{3}$	$-\frac{3\pi}{4}$	$-\frac{5\pi}{6}$	$-\pi$

y	1	$\frac{\sqrt{3}}{2}$	$\frac{\sqrt{2}}{2}$	0.5	0	-0.5	$-\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{3}}{2}$	-1

x	$-\frac{7\pi}{6}$	$-\frac{5\pi}{4}$	$-\frac{4\pi}{3}$	$-\frac{3\pi}{2}$	$-\frac{5\pi}{3}$	$-\frac{7\pi}{4}$	$-\frac{11\pi}{6}$	-2π
y	$-\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{2}}{2}$	-0.5	0	0.5	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1

x	$-\frac{19\pi}{6}$	$-\frac{85\pi}{4}$	$-\frac{37\pi}{3}$	$-\frac{30000\pi}{2}$	$-\frac{37040\pi}{3}$	$-\frac{214103\pi}{4}$	$-\frac{416603\pi}{6}$	-857238π
y	$-\frac{\sqrt{3}}{2}$	$-\frac{\sqrt{2}}{2}$	-0.5	0	0.5	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$	1



Рассмотренная область от -inf до +inf

Задание 1. Программные ограничения для функции

Числа в компьютере должны представляться конечными числами и, они в свою очередь обладают конечной точностью

Будем использовать для переменных x, y – тип данных Double

Абсолютный максимум: $Double.MAX_VALUE \approx 1.797631348623157 * 10^{308}$

Абсолютный минимум: $-Double.MAX_VALUE \approx -1.797631348623157 * 10^{308}$

Минимальное положительное значение: $Double.MIN_VALUE \approx 4.9 * 10^{-324}$

Максимальное отрицательное значение: $-Double.MIN_VALUE \approx -4.9 * 10^{-324}$

Поэтому область допустимых значений для x: [-Double.MAX_VALUE; -Double.MIN_VALUE] v {0} v [Double.MIN_VALUE; Double.MAX_VALUE]

А область допустимых значений для y: [-1; -Double.MIN_VALUE] v {0} [Double.MIN_VALUE; 1].

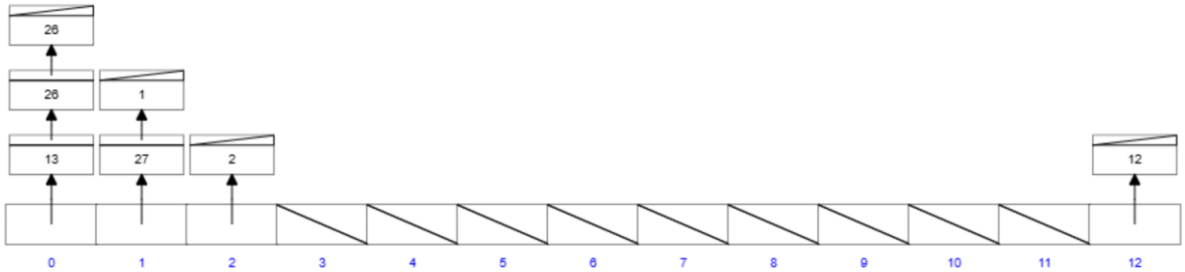
Если значение $x \in [-\infty;\ -Double.MAX_VALUE)\ v\ (Double.MAX_VALUE;\ +\infty]$, то вызываем ошибку “The number is infinite”

Если значение $x, y \in [-Double.MIN_VALUE;\ -0)\ v\ (0; Double.MIN_VALUE]$, то считаем, что данное значение равно 0

x	10^{200}	-10^{200}	10^{309}	-10^{309}	0		$5 * 10^{-325}$		$-5 * 10^{-325}$	
y						0		$5 * 10^{-325}$		$-5 * 10^{-325}$
Есть ошибка?	Нет	Нет	Есть	Есть	Нет	Нет	Нет	Нет	Нет	Нет

Задание 2.

Функция Insert для чисел



Можем заметить, что вставление в HashMap и определение нужного bucket, определяется значением вводимого поля и взять остаток на деление на 13

Если числа попадает в один и тот же backet, то добавляем число в этот bucket, поиск начинаем с этого числа

Функция Insert для строк

Строки переводятся в число с помощью алгоритма, в дальнейшем вставка осуществляется аналогично как для чисел.

Алгоритм перевода строки в число

Для каждого символа c в строке выполняется операция:

h = (h << 4) XOR code(c), где code(c) — числовой код символа (например, ASCII).

Функция Delete

Удаление проходит таким образом:

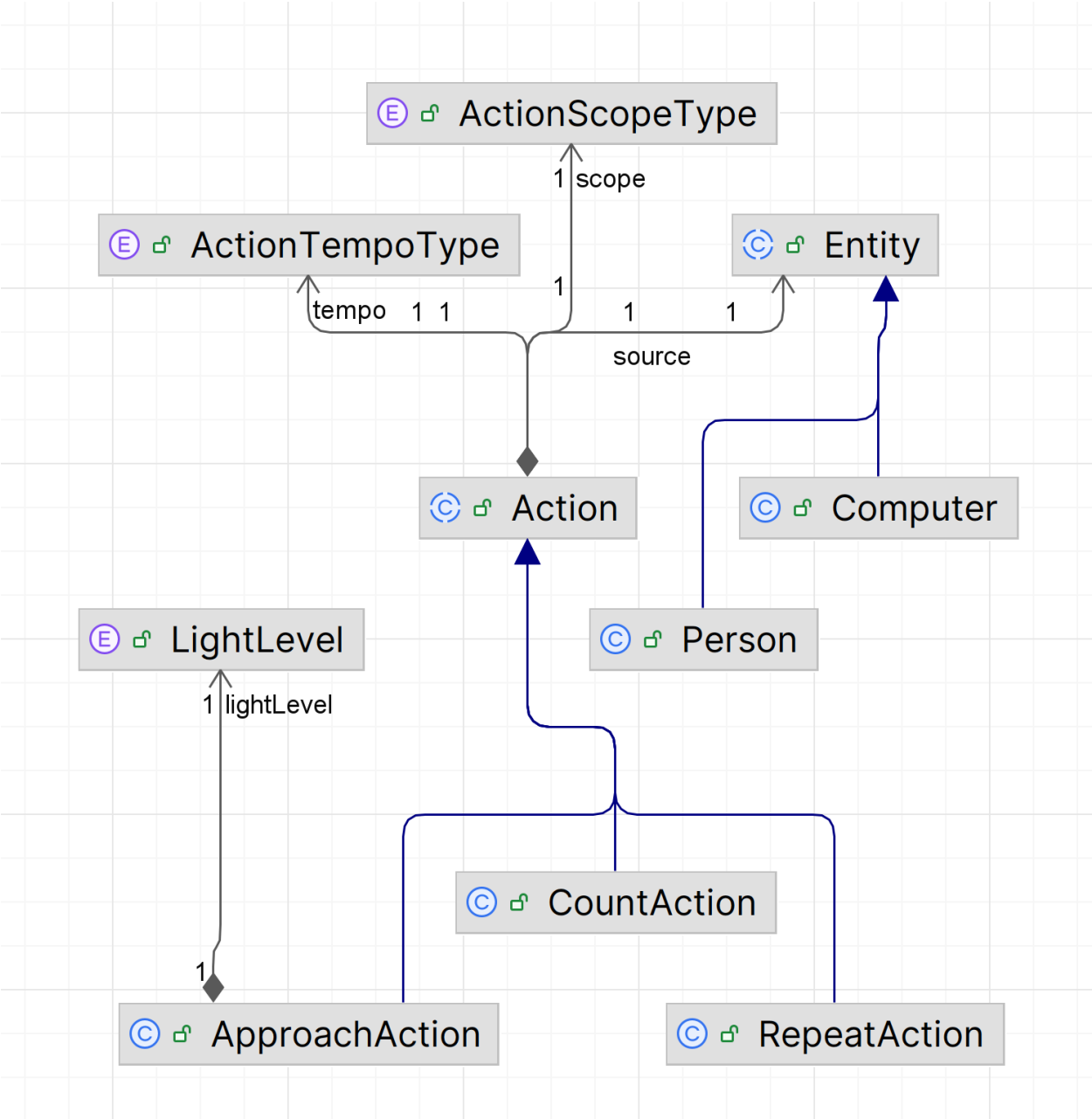
1. Ищем нужный bucket
2. Последовательно ищем элемент, который должен находиться в этом бакете
3. Если его нет, то ничего не удаляем, если есть то удаляем только один

Функция Find

Поиск проходит следующим образом:

1. Ищем hash для обнаружения нужного bucket
2. Последовательно ищем конкретный элемент в этом бакете
3. Если найден, возвращаем true, в противном случае false

Задание 3. Доменная модель для заданного текста и тестовое покрытие для неё



Форд продолжал считать вслух. Это одно из самых агрессивных действий, которые вы можете применить к компьютеру, равносильное тому, чтобы медленно приближаться в темноте к человеку, повторяя: "Умри... умри... умри..."

Я выделил в рамках доменной модели сущности: компьютер и человек

В качестве действий выделил такие: подсчет чисел, приближение и повтор слов

В качестве типов: ActionScopeType, ActionTempoType и LightLevel

Тесты реализовал для каждого действия

ApproachActionTest (“Приближение”)

В данном классе тестируется следующее:

- Влияние освещённости на уровень стресса при приближении. Чем темнее, тем страшнее
- Влияние скорости на уровень стресса при приближении. Чем быстрее, тем страшнее
- Проверка на то, что действие “приближение” не оказывает влияние на уровень стресса у компьютера

CountActionTest (“Подсчет чисел”)

В данном классе тестируется следующее:

- Влияние количество чисел на уровень стресса компьютера. Чем больше чисел, тем больше стресса
- Влияние скорости на уровень стресса. Чем медленнее считать, тем страшнее
- Проверка на то, что действие “подсчет чисел” не оказывает влияние на уровень стресса у человека
- Проверка на то, что если не озвучивать подсчет чисел, то компьютеры не боятся

RepeatActionTest (“Повтор текста”)

В данном классе тестируется следующее:

- Повторение слов от компьютера страшнее, чем от человека
- Влияние количество повторов на уровень стресса
- Влияние скорости повтора на уровень стресса
- Проверка на то, что действие “повтор текста” не оказывает влияние на уровень стресса у компьютера

DomainModelTest

- Проверка равносильности уровню стресса сущности “компьютер” и “человек”, описанных в тексте, благодаря определенному порядку действий (Надуманный кейс)

Исходный код или ссылка на него

https://github.com/danp1t/TPO_LABS/tree/main/lab1

Вывод

В данной лабораторной работе я вспомнил как проводить модульное тестирование и на что необходимо уделять внимание при составлении кейсов. Осознал, насколько метод TTD удобен при написании функций и позволяет тебе проверить её корректность на множестве тестов, а также осознал полезность регрессионного тестирования, ведь при одной правке, может код сломаться в неожиданном месте, а без тестов ты бы этого не заметил. В общем, данная лабораторная работа помогла мне осознать, как хорошо сделанные тесты могут повысить качество и надежность работы программы.